
The Publishing Accelerator Toolkit (PacTk)

Amy Neustein, Ph.D.
CEO and Founder
Linguistic Technology Systems
Fort Lee, NJ
917-817-2184
amy.neustein@verizon.net

A Collection of Software, Compiler, and Runtime Tools for Managing and Visualizing Multimedia Data Sets

Overview

In contemporary publishing, the standard workflow includes **XML**-based formats such as **JATS** or from which are compiled reader-visible documents such as **PDFs**. These intermediate files are distinct from those that authors submit directly, and they can be computationally accessed via generic **XML**-oriented protocols such as (Simple **API** for **XML**), , and (scripting extension). Unfortunately, **XML** itself is not explicitly built for representing textual data, and examining **JATS** or files via generic **XML** technology renders certain query and editing tasks difficult or impossible to implement. The **PDF** format is similarly opaque because internal **PDF** representations of document text include many anomalies due to ligatures, hyphenation, font specs, and other layout-related discrepancies between **XML** inputs and user-visible output.

Given the limitations of **XML** for natural-language documents, numerous competing formats have been developed that provide alternatives or extensions to **XML** structures, including **TEI** (Text Encoding Initiative), (Text-as-Graph Markup Language), **BIOC** (a biomedical text mining standard defined by PubMed Central), and Research Object Bundles (a Semantic-Web based specification that converts both text and data sets). With a proliferation of different text-encoding standards, query and traversal protocols that are developed for one specific format (e.g., **JATS** and its derivatives) cannot be directly applied to others. For example, the **XML** parent-child element relationship is replaced in with two different containment styles (logical and extensional), such that **DOM** and event-based parsers targeted at the simpler **XML** hierarchy must be reimplemented for the system. From the perspective of , text-encoding evinces a spectrum of distinct *topomorphic regimes* where a proliferation of incompatible document structures prohibits the emergence of common query, traversal, and metadata formats.

One important variation amongst text-encoding formats is the level of granularity for most textual elements. In **JATS**, , and most **JSON** encodings, the fundamental discursive unit is the individual paragraph. Elements on smaller scales (such as sentences or block quotes) are notated haphazardly at best. **L^AT_EX** employs reasonable sentence-boundary heuristics (that can be overridden via macros when necessary), but sentence objects are not expressed in any systematic manner (e.g., via auxiliary files) except for intermediate spacing. In **TEI** and **BIOC**, individual sentences *can* be represented, but this is not mandatory (thus **s** and **sentence** tags are available but not guaranteed in any particular document). By contrast, is explicitly built around sentence-level units, specifically a “topomorphic sentence-level object model” (). When all regular text in a natural-language document is explicitly contextualized in a delineated sentence object, many editing, indexing, and user-interface tasks become feasible (as outlined in the next section).

In contrast to the other formats mentioned here, is not a single file format but rather a collection of code libraries and classes, with data types representing specific textual elements (sentences, paragraphs, **PDF** pages, annotations and comment-boxes, etc.). Query and traversal protocols modeled via , **DOM**, etc., become expressed by object methods and operators exposed to **C++** code, for example (is

thus “topomorphic” in the sense that navigation between objects is determined by structural properties of divergent document formats, that manifest in competing notions of individuation for individual structure-sites and their interrelationships). In short, the object system synthesizes distinct structuring protocols found in competing formats such as **JATS**, , and **BIOC**, allowing publishers’ workflows to including each of these formats (and others) without sacrificing compatibility. is particularly aimed at publishers who feel constrained by the limitations of **JATS** or but are reluctant to try alternatives (, **BIOC**, etc.) due to minimal software-ecosystem support and the lack of standardized and **DOM**-style interfaces.

The remainder of this document will discuss how can streamline editing, indexing, and document-preparation tasks, as well as improve User Experience for human readers. Later sections will also address how can contribute to broader projects where text documents are published together with data sets and/or multimedia presentations. Overall, presents a unified model and coding interface for text queries operating in diverse contexts, including **PDF** viewers, search engines, and document-preparation code. Similarly, software employing to navigate through document objects can leverage interop metadata across different formats, such as **XML** elements and **PDF** page coordinates, that are typically unavailable in other publishing tools.

Features for Editing, Indexing, and PDF Viewers

The computational foundation of is the collection of classes that represented individual sentences and other textual elements in the family (**PDF** pages, annotations, footnotes, block quotes, edits or changes across document versions, etc.). In order to initialize these objects, provides a distinct input format called , or “grounded” . The term “grounded” reflects how certain structures-sites may be specifically marked as serializing a typed object: i.e., tags, attributes, or text nodes in the neighborhood of a site supply the data fields needed to initialize an object-instance during deserialization. In other respects follows most of the principles advocated by , with novel extensions; for example, encourages the exploitation of Unicode Private Use Planes to disambiguate visibly similar but logically distinct characters/glyphs (e.g., periods may embody punctuation, acronyms, or abbreviations). In source files, is loosely based on Markdown, so authors who use Markdown in more informal contexts (e.g., social media, chat rooms, **GIT** documentation, etc.) will recognize many of the styling commands. sources might be provided by authors directly, or derived from author inputs by digital copy editors or others on the publishing workflow. To support , provides a new **C++** parsing library that is readily extensible (more so than parsers for **XML**, **JSON**, and so forth), allowing publishers to customize the format with new tags, syntax, and data structures specific to their in-house styles and disciplinary focus areas (e.g., specialized tags/data for different scientific fields — chemistry, physics, medicine, etc. — or whatever subject-areas are addressed by individual publishers, journals, and book series). Alternatively, objects can be compiled from sources such as **JATS** or directly.

For workflows that feature **JATS**, , or **L^AT_EX**, object bundles would be used as adjunct data sources through which edits, queries, and other computational artifacts would be routed. data can then be mapped both to archiving formats (like **XML**) and camera-ready generators (like **L^AT_EX**), but with a common background these two different representational facets may be integrated (for example, **XML** elements mapped to corresponding **PDF** page coordinates). In effect, serves to triangulate human-visible presentations (like **PDF**) with behind-the-scenes **JATS**-style formats. Analyses that are currently implemented against **XML** structures may be more efficiently coded via objects, with concrete benefits in areas such as copy-editing and indexing.

PacTk for Editing

has an advanced query framework to help find specific locations in text where edits may be warranted (e.g., to comport with journal or book-series styles) and to correctly review and process such edits. Here are some features:

Contextual Filters for Text Locations Often it is useful to restrict the scope of text searches to characters with specific presentation features (italics, alternative typefaces, citations), discursive roles (inline/block quotes, footnote text, section/subsection titles), or semantic significance (acronyms, annotations, proper/place names).

Implementing such filters via /tag restrictions can be inaccurate, because relevant filter criteria may be orthogonal to **DOM** hierarchies. For example, **XML** or **L^AT_EX** code may wish to wrap elements such as footnotes or block quotes in custom environments, to ensure desired pre- or post-processing. Content that is functionally footnote or quoted text, therefore, may not necessarily be indicated with those specific tag names. The system allows programmers to define lists of boolean parameters that apply to text-sequences and then activate those flags automatically in specific tag contexts.

In general, supports queries where ordinary textual restrictions may be augmented with filters for a wide variety of layout and discursive contexts. Such queries could then be used for editing tasks that should be localized to specific contexts, or where a user wants to find a specific piece of text known to have a given context.

Sentence-Level Queries Siting queries relative to sentence boundaries can significantly improve precision and express query rationales more succinctly. For instance, the proper punctuation around a word or phrase often depends on whether that content appears at

the beginning, middle, or end of a sentence. Some query expressions may similarly depend on stipulating that two or more search targets (e.g., two proper names) occur in the same sentence.

Because recognizes almost all text as contained inside sentence objects, queries dependent on sentence boundaries can be directly evaluated on data (such boundaries are unambiguously notated via input sources, rather than detected approximatively using Natural Language Processing). likewise models “topomorphic” and potentially-navigational relations between sentences, such as between adjacent siblings of parent containers (typically paragraphs) and also nestings where a multi-sentence footnote, for example, occurs in a parent-sentence context, or where a block quotes functions in part as a logical child of the sentence that introduces it.

PDF Page Coordinates Human readers post-publication — plus authors and copy-editors beforehand — typically view documents in PDF format for normal reading, even if edits/changes are made on intermediary files. As a result, a bitmapped page-image is the primary medium through which users view single publication pages. Failure to interoperate these images with structured text formats is a significant hindrance to efficient document preparation and editing.

By design, supports post-processing algorithms within pipelines whose formats can generate PDF-coordinate metadata, such as **LaTeX pgfsavepos** (and auxiliary **write**) commands. Page coordinates for significant discursive locations (e.g., sentence and paragraph boundaries, footnote markers, keyphrases and named entities, citations) can thereby be exposed to PDF viewers which could thereby implement novel front-end features such as overlays, context menus, and dialog windows positioned nearby relevant content. For example, a context menu action may include copying the full text of a current sentence to clipboard, based on the screen position where the menu is activated; semitransparent overlays could similarly show the extent of individual sentences during mouseover events.

PDF Comments and Annotations In addition to sentence objects, classes model data specifically relevant to the PDF front-end milieu, including single pages, and PDF comments/annotations. Objects such as comment-boxes may thereby be leveraged systematically in an editing/review pipeline.

In particular, supports construction of hashtag- or handle-style controlled vocabularies that can annotate PDF comments introduced by copy editors or typesetters. Many modifications considered during the editing process follow some common patterns, such as proper use of commas around subordinate clauses, or correct formatting of ellipses and bracketed content within inline/block quotes. Assuming that editors consistently include notations for recurring patterns, code can filter queries specifically to PDF comment boxes which indicate a specific editing pattern. Index codes within PDF comments can likewise indicate which individual sentence surrounds the annotation, so that the PDF view may be cross-referenced against input sources, which facilitates the review and processing of edit-related comments.

For similar reasons, supports abstractions such as XML custom character entities and LaTeX macros. Many familiar textual components — ellipses, m- and n-dashes, quotation and footnote marks, etc. — are typically represented not via their visible characters but rather through standardized codes or glyphs. The problem is that systematic mappings are lacking when mixing different filetypes (e.g., LaTeX and XML, such as JATS or) in a single workflow. has a flexible abstraction system that allows character and macro codes to propagate across different generated files. A related problem is that abstraction codes should generally be opaque to human users in contexts such as comment boxes and copy-paste between software applications. therefore supports declarations for how custom entities/macros and Unicode Private Use Area codepoints should be rendered in plain-text contexts as well as structured formats such as XML and LaTeX.

meta-index

Compiler and Virtual Machine (VM) Components

Constraint Declarations and Other VM Use-Cases

Cross-Referencing Across Diverse Multimedia Content Types

Texture Annotations

Software, Cloud, and GUI Modules



Data Sets, Queries, and Database Architecture

Some References

Video Links

For more information, please see our software demo vidoes: